Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №1**
По дисциплине : «Модели решения задач в ИС»
По теме : «Адаптивный шаг обучения в однослойном линейном персептроне»

**Выполнил:**
студент 3 курса
группы ИИ-26
Заруцкий В.Я.
**Проверил:**
Адренко К.В.

Брест 2026

**Цель работы:** изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1.

## Вариант 4:

| $x_1$ | $x_2$ | e |
|---|---|---|
| 4 | 1 | 1 |
| -4 | 1 | 1 |
| 4 | -1 | 1 |
| -4 | -1 | 0 |

## Ход работы :

## Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

def load_custom_dataset():
    data = [
        [4, 1, 1],
        [-4, 1, 1],
        [4, -1, 1],
        [-4, -1, 0]
    ]
    X = np.array([[p[0], p[1]] for p in data], dtype=float)
    y = np.array([p[2] for p in data], dtype=int)
    y = np.where(y == 0, -1, 1)
    return X, y

def step_sign(x):
    return 1 if x >= 0 else -1

class ADALINESequential:
    def __init__(self, input_size, lr=0.01):
        self.w = np.zeros(input_size, dtype=float)
        self.b = 0.0
        self.lr = lr

    def predict_raw(self, x):
        return np.dot(self.w, x) + self.b

    def predict(self, x):
        return step_sign(self.predict_raw(x))

    def train_fixed(self, X, y, epochs=100, E_eps=1e-6, shuffle=True):
        n = X.shape[0]
        E_history = []
        for epoch in range(epochs):
            if shuffle:
                idx = np.random.permutation(n)
            else:
                idx = np.arange(n)
            E = 0.0
            for i in idx:
                xi = X[i]
                yi = y[i]
                raw = self.predict_raw(xi)
                err = yi - raw
                self.w += self.lr * err * xi
                self.b += self.lr * err
                E += 0.5 * err**2
            E_history.append(E)
            print(f"[Fixed] Эпоха {epoch+1:3d}: E = {E:.6f}")
            if E <= E_eps:
                break
        return E_history

    def train_adaptive_236(self, X, y, epochs=100, E_eps=1e-6, shuffle=True):
        n = X.shape[0]
        E_history = []
        for epoch in range(epochs):
            if shuffle:
                idx = np.random.permutation(n)
            else:
                idx = np.arange(n)
```

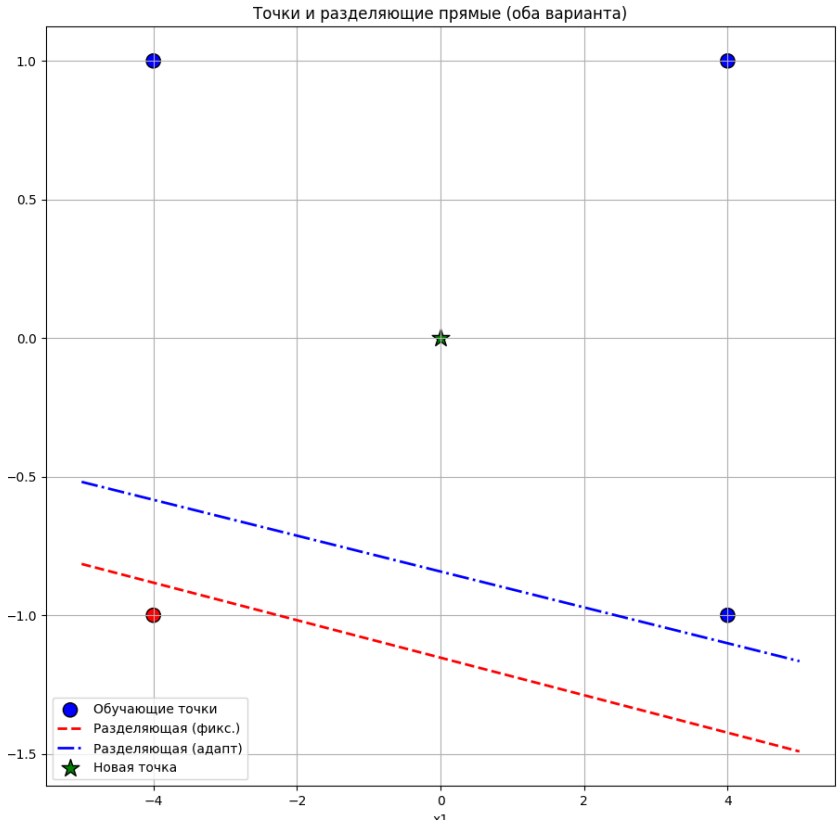
```
            E = 0.0
            for i in idx:
                xi = X[i]
                yi = y[i]
                raw = self.predict_raw(xi)
                err = yi - raw
                denom = 1.0 + np.dot(xi, xi)
                alpha_t = 1.0 / denom
                self.w += alpha_t * err * xi
                self.b += alpha_t * err
                E += 0.5 * err**2
            E_history.append(E)
            print(f"[Adaptive] Эпоха {epoch+1:3d}: E = {E:.6f}")
            if E <= E_eps:
                break
        return E_history

    def decision_boundary(self, x_vals):
        if abs(self.w[1]) < 1e-12:
            return np.full_like(x_vals, np.nan)
        return (-(self.w[0] * x_vals + self.b) / self.w[1])

def run_experiments():
    X, y = load_custom_dataset()
    max_epochs = 100
    E_eps = 1e-6

    model_fixed = ADALINESequential(input_size=2, lr=0.05)
    E_fixed = model_fixed.train_fixed(X, y, epochs=max_epochs, E_eps=E_eps, shuffle=True)

    model_adapt = ADALINESequential(input_size=2, lr=None)
    E_adapt = model_adapt.train_adaptive_236(X, y, epochs=max_epochs, E_eps=E_eps, shuffle=True)

    fig, axes = plt.subplots(1, 2, figsize=(14,6))

    ax = axes[0]
    ax.plot(range(1, len(E_fixed)+1), E_fixed, 'r-o', label='Фиксированный шаг (0.05)')
    ax.plot(range(1, len(E_adapt)+1), E_adapt, 'b-s', label='Адаптивный')
    ax.set_xlabel("Эпоха p")
    ax.set_ylabel("Суммарная ошибка E(p)")
    ax.set_title("Сходимость: фиксированный vs адаптивный")
    ax.legend()
    ax.grid(True)

    ax2 = axes[1]
    colors = ['red' if label == -1 else 'blue' for label in y]
    ax2.scatter(X[:,0], X[:,1], c=colors, edgecolors='k', s=120, label='Обучающие точки')

    x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
    x_vals = np.linspace(x_min, x_max, 200)

    y_fixed = model_fixed.decision_boundary(x_vals)
    if not np.isnan(y_fixed).all():
        ax2.plot(x_vals, y_fixed, 'r--', linewidth=2, label='Разделяющая (фикс.)')

    y_adapt = model_adapt.decision_boundary(x_vals)
    if not np.isnan(y_adapt).all():
        ax2.plot(x_vals, y_adapt, 'b-.', linewidth=2, label='Разделяющая (адапт)')

    for xi, yi in zip(X, y):
        pred_f = model_fixed.predict(xi)
        pred_a = model_adapt.predict(xi)


    new_point = np.array([0.0, 0.0])
    pred_f_new = model_fixed.predict(new_point)
    pred_a_new = model_adapt.predict(new_point)
    ax2.scatter(new_point[0], new_point[1], c='green', s=200, marker='*', edgecolors='k',
            label=f'Новая точка')

    ax2.set_xlabel("x1")
    ax2.set_ylabel("x2")
    ax2.set_title("Точки и разделяющие прямые (оба варианта)")
    ax2.legend()
    ax2.grid(True)

    plt.tight_layout()
    plt.show()

    preds_fixed = np.array([model_fixed.predict(x) for x in X])
    acc_fixed = np.mean(preds_fixed == y) * 100
    preds_adapt = np.array([model_adapt.predict(x) for x in X])
    acc_adapt = np.mean(preds_adapt == y) * 100

    print("Фиксированный шаг: веса =", model_fixed.w, "bias =", model_fixed.b, f"Точность = {acc_fixed:.1f}%")
    print("Адаптивный: веса =", model_adapt.w, "bias =", model_adapt.b, f"Точность = {acc_adapt:.1f}%")
```

```
if __name__ == "__main__":
    run_experiments()
```

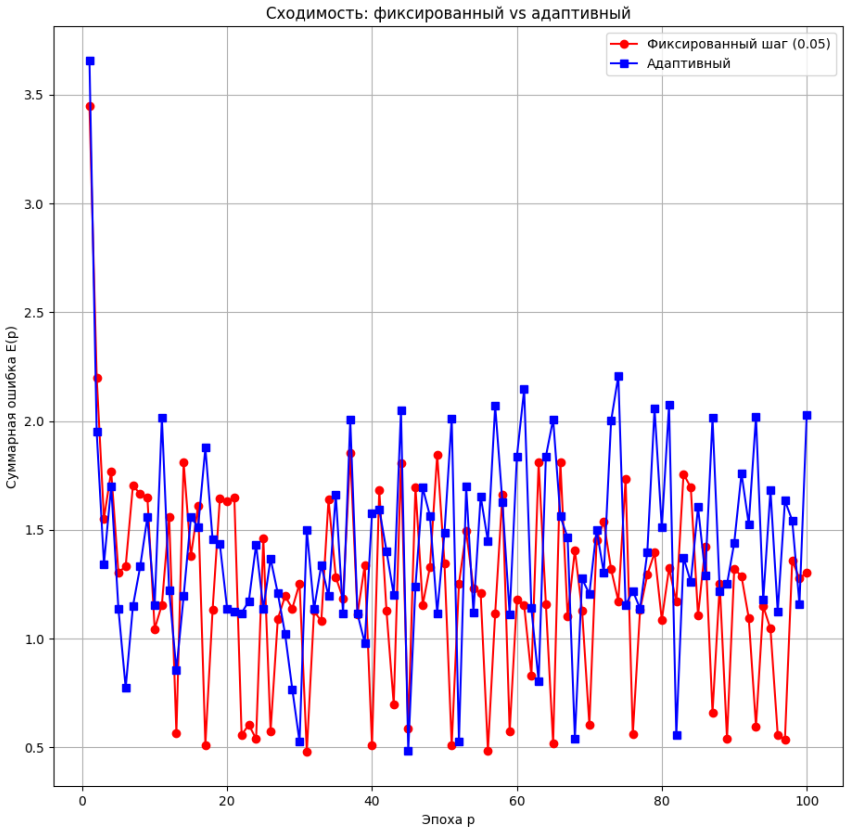
## Результат тестирования:

```
[Fixed] Эпоха   1: E = 3.447952
[Fixed] Эпоха   2: E = 2.200845
[Fixed] Эпоха   3: E = 1.551227
[Fixed] Эпоха   4: E = 1.769149
[Fixed] Эпоха   5: E = 1.303246
[Fixed] Эпоха   6: E = 1.333464
[Fixed] Эпоха   7: E = 1.703860
[Fixed] Эпоха   8: E = 1.665007
[Fixed] Эпоха   9: E = 1.649123
[Fixed] Эпоха  10: E = 1.044347
[Fixed] Эпоха  20: E = 1.630379
[Fixed] Эпоха  30: E = 1.253822
[Fixed] Эпоха  40: E = 0.510034
[Fixed] Эпоха  50: E = 1.347716
[Fixed] Эпоха  60: E = 1.179152
[Fixed] Эпоха  70: E = 0.603361
[Fixed] Эпоха  80: E = 1.086566
[Fixed] Эпоха  90: E = 1.322013
[Fixed] Эпоха 100: E = 1.305154
[Adaptive] Эпоха   1: E = 3.655832
[Adaptive] Эпоха   2: E = 1.949536
[Adaptive] Эпоха   3: E = 1.342985
[Adaptive] Эпоха   4: E = 1.699620
[Adaptive] Эпоха   5: E = 1.136954
[Adaptive] Эпоха   6: E = 0.774268
[Adaptive] Эпоха   7: E = 1.149289
[Adaptive] Эпоха   8: E = 1.331178
[Adaptive] Эпоха   9: E = 1.558941
[Adaptive] Эпоха  10: E = 1.152407
[Adaptive] Эпоха  20: E = 1.138898
[Adaptive] Эпоха  30: E = 0.525505
[Adaptive] Эпоха  40: E = 1.574301
[Adaptive] Эпоха  50: E = 1.486114
[Adaptive] Эпоха  60: E = 1.836657
[Adaptive] Эпоха  70: E = 1.204975
[Adaptive] Эпоха  80: E = 1.510723
[Adaptive] Эпоха  90: E = 1.440591
[Adaptive] Эпоха 100: E = 2.030070
Фиксированный шаг: веса = [0.03275474 0.48417013] bias = 0.5582688583870692 Точность = 100.0%
Адаптивный: веса = [0.04085961 0.63157058] bias = 0.5318678620780268 Точность = 100.0%
```

## График с разделяющей поверхностью:




## График изменения ошибки:




**Вывод:** Изучил алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовал модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравни скорость сходимости с классическим алгоритмом из Лабораторной работы №1.